Problema A. Planejamento da Grade Curricular

Entrada de dados:	`entrada padrão`
Saída de dados:	`saída padrão`
Limite de tempo:	3 segundos
Limite de memória:	256 megabytes

Uma universidade está organizando a oferta de disciplinas para seus estudantes. Cada uma das n disciplinas possui um código numérico de 0 a $n-1$. Para evitar que os estudantes se matriculem em disciplinas sem possuir os conhecimentos necessários, algumas delas exigem que outras sejam concluídas anteriormente.

A sua tarefa é definir uma ordem para as n disciplinas de forma que, se o aluno seguir essa ordem, ele conseguirá finalizar todas as disciplinas respeitando os pré-requisitos de cada uma, ou seja, uma disciplina só aparece na ordem após todos os seus pré-requisitos terem aparecido antes.

Entrada

A primeira linha do caso de teste é composta por dois inteiros n, indicando a quantidade de disciplinas, e m, indicando a quantidade de dependências entre as disciplinas. Seguem-se m linhas. Cada linha contém dois inteiros i e j, indicando que a disciplina i é pré-requisito da disciplina j.

É garantido que

- $1 \leq n \leq 10^5$, $0 \leq m \leq 3 \cdot 10^6$;
- Para cada par de inteiros i e j a partir da segunda linha, temos que $0 \leq i \neq j \leq n-1$; e
- Não existem dependências cíclicas entre as disciplinas.

Saída

A saída deve conter apenas uma permutação dos inteiros de 0 a $n-1$, separados por espaço, de forma que cada disciplina apareça na ordem depois de todos os seus pré-requisitos. É garantido haver **exatamente** uma ordem que satisfaz a condição.

Lembre-se de que cada linha da saída, inclusive a última, deve terminar com um "\n".

Exemplos

Entrada	Saída
3 3 0 2 2 1 0 1	0 2 1
4 4 0 3 3 2 2 1 3 1	0 3 2 1
5 7 3 2 3 1 1 2 0 3 1 4 0 4 2 4	0 3 1 2 4

Problema B. Rotas Seguras

Entrada de dados:	`entrada padrão`
Saída de dados:	`saída padrão`
Limite de tempo:	1 segundo
Limite de memória:	256 megabytes

Uma equipe de exploração está mapeando as rotas de uma região desconhecida para identificar quais pontos podem ser visitados sem correr o risco de ficar presa em um ciclo sem saída. O mapa é representado por um grafo direcionado com n vértices, numerados de 0 a $n-1$, e m arcos, no qual cada arco (i, j) indica que é possível ir do ponto i para o ponto j.

O grafo é representado por uma lista de adjacências, G, onde $G[i]$ contém os vértices que podem ser alcançados diretamente a partir do vértice i.

Um vértice é considerado sorvedouro quando seu grau de saída é zero. Já um vértice é considerado seguro quando, independentemente das escolhas feitas durante a exploração, qualquer caminho iniciado a partir dele eventualmente chega a um vértice sorvedouro. Em outras palavras, não pode existir um caminho de um vértice seguro para um ciclo direcionado. Sua tarefa é determinar todos os vértices seguros.

Entrada

A primeira linha do caso de teste é composta por dois inteiros n, indicando a quantidade de disciplinas, e m, indicando a quantidade de dependências entre as disciplinas. Seguem-se m linhas. Cada linha contém dois inteiros i e j, indicando que a disciplina i é pré-requisito da disciplina j.

É garantido que

- $1 \leq n \leq 10^5$, $0 \leq m \leq 10^6$; e
- Para cada par de inteiros i e j a partir da segunda linha, temos que $0 \leq i \neq j \leq n-1$.

Saída

A saída contém apenas uma linha com inteiros. O primeiro inteiro é a quantidade k de vértices seguros do digrafo. Em seguida, a linha deve conter a sequência ordenada de forma crescente de todos os vértices seguros do digrafo. Os inteiros da linha de saída devem estar todos separados por espaço.

Lembre-se de que cada linha da saída, inclusive a última, deve terminar com um "\n".

Exemplos

Entrada	Saída
3 3 0 2 2 1 0 1	3 0 1 2
4 4 0 3 3 2 2 1 1 3	0
5 7 0 3 0 2 3 2 2 3 3 1 2 1 1 4	2 1 4

Problema C. Hierarquia e Experiência

Entrada de dados:	`entrada padrão`
Saída de dados:	`saída padrão`
Limite de tempo:	1 segundo
Limite de memória:	256 megabytes

Uma empresa possui n funcionários, numerados de 0 a $n-1$. Alguns funcionários ocupam cargos hierarquicamente superiores a outros, e cada funcionário tem sua experiência quantificada entre 0 e $n-1$, sendo única para cada funcionário. A sua tarefa é identificar, para cada funcionário i, quem é o funcionário mais experiente entre o próprio i e todos aqueles que estão em um cargo de nível superior ao cargo de i.

Entrada

A primeira linha do caso de teste é composta por dois inteiros separados por espaço: n, indicando a quantidade de funcionários, e m, indicando a quantidade de dependências hierárquicas entre eles. A segunda linha contém n inteiros separados por espaço:

$$\texttt{C}[0]\ \texttt{C}[1]\ \ldots\ \texttt{C}[n-1]$$

onde $\texttt{C}[i]$ representa a experiência do funcionário i.

Seguem-se m linhas. Cada linha contém dois inteiros i e j, indicando que o funcionário i possui um cargo mais elevado hierarquicamente do que o funcionário j. As relações hierárquicas são consistentes, ou seja, não teremos uma situação em que i tem um cargo superior a j e vice-versa, mas nem todos os funcionários necessariamente possuem uma relação hierárquica entre si.

É garantido que

- $1 \leq n \leq 10^5$, $0 \leq m \leq 10^6$;
- para cada $0 \leq i \leq n-1$, $0 \leq C[i] \leq n-1$;
- para cada $0 \leq i \neq j \leq n-1$, $C[i] \neq C[j]$; e
- para cada par de inteiros i e j a partir da segunda linha, temos que $0 \leq i \neq j \leq n-1$;
- não há relações hierárquicas cíclicas.

Saída

A saída deve conter apenas uma linha com n inteiros. O i-ésimo inteiro deve ser o funcionário mais experiente entre o próprio i e aqueles que ocupam um cargo de nível superior ao cargo de i.

Lembre-se de que cada linha da saída, inclusive a última, deve terminar com um "\n".

Exemplos

Entrada	Saída
3 3 1 0 2 0 2 2 1 0 1	0 2 2
4 4 0 2 1 3 1 0 1 3 0 2 3 2	1 1 3 3
4 2 0 1 2 3 0 1 3 2	0 1 3 3

Problema D. Labirinto de 6 dimensões

Entrada de dados:	`entrada padrão`
Saída de dados:	`saída padrão`
Limite de tempo:	0.3 segundos
Limite de memória:	256 megabytes

Você está em um labirinto de 6 dimensões. Cada dimensão possui 10 posições possíveis, identificadas pelos algarismos '0', '1', '2', '3', '4', '5', '6', '7', '8' e '9'. Em cada movimento, você pode avançar ou retroceder uma posição em exatamente uma das seis dimensões. Por exemplo, da posição 345678, você pode alcançar, com um movimento, as posições 245678, 335678, 355678, 346678, 345578, 345778, 345668 ou 345679, dentre outras.

O labirinto possui uma propriedade peculiar: **ele é circular**. Isso significa que, ao avançar a partir da posição '9' em uma das 6 dimensões, você retorna à posição '0', e, ao retroceder a partir da posição '0', você chega à posição '9'.

O labirinto possui n posições que não devem ser visitadas. Caso você alcance qualquer uma delas, ficará preso e não poderá continuar a sua jornada. Inicialmente, você se encontra na posição 000000. A sua tarefa é determinar o número mínimo de movimentos necessários para alcançar a saída t do labirinto a partir da posição inicial.

Entrada

A primeira linha do caso de teste é composta por um inteiro n e uma string t, separados por espaço, indicando a quantidade de posições que não devem ser visitadas e a saída do labirinto, respectivamente.

Se n for maior do que zero, o caso de teste possui uma segunda linha. A segunda linha é composta por n strings separadas por espaços

$$\mathtt{p}[0]\ \mathtt{p}[1]\ \ldots\ \mathtt{p}[n-1]$$

representando as posições que não devem ser visitadas.

É garantido que

- $0 \leq n \leq 10^5$; e
- t e $\mathtt{p}[i]$, para cada $0 \leq i \leq n-1$, são strings com exatamente 6 caracteres entre '0' e '9'.

Saída

A saída deve conter apenas uma linha contendo a quantidade mínima de movimentos necessários para alcançar a saída t. Caso não seja possível chegar à saída sem passar por uma posição que não deve ser visitada, a linha deve conter apenas o inteiro -1. A posição inicial 000000 e/ou a saída t do labirinto podem ser posições que não devem ser visitadas, caso no qual a linha de saída deve ser -1.

Lembre-se de que cada linha da saída, inclusive a última, deve terminar com um "\n".

Exemplos

Entrada	Saída
1 000200 000100	4
0 111111	6
2 110000 100000 010000	4

Problema E. Labirinto Decrescente

Entrada de dados:	`entrada padrão`
Saída de dados:	`saída padrão`
Limite de tempo:	0.3 segundos
Limite de memória:	256 megabytes

Você está explorando um labirinto representado por uma matriz quadrada de dimensões $n \times n$, partindo da célula $(0, 0)$. Cada célula contém um valor que indica a altitude daquele ponto do labirinto. Para chegar à saída, localizada no canto inferior direito (célula $(n-1, n-1)$), você deve seguir um caminho que não contém subidas: a altitude de cada nova célula visitada deve ser **menor ou igual** à da célula anterior.

Em cada movimento, você pode se deslocar para qualquer uma das oito células vizinhas, as células que compartilham um lado ou um canto, desde que permaneça dentro dos limites do labirinto. Sua tarefa é achar a quantidade mínima de movimentos necessários para ir da entrada à saída do labirinto.

Entrada

A primeira linha do caso de teste é composta por um inteiro n, indicando as dimensões da matriz. Depois, seguem-se n linhas, correspondendo à matriz A que representa o labirinto.

Cada linha $0 \leq i \leq n-1$ é composta por n inteiros separados por espaços

$$\texttt{A}[i][0]\ \texttt{A}[i][1]\ \ldots\ \texttt{A}[i][n-1]$$

representando cada linha de A.

É garantido que

- $1 \leq n \leq 1000$; e
- para cada $0 \leq i, j \leq n-1$, $0 \leq \texttt{A}[i][j] \leq 10^9$.

Saída

A saída deve conter apenas uma linha contendo a quantidade mínima de movimentos necessários para, a partir da entrada na posição $(0, 0)$, alcançar a saída do labirinto na posição $(n-1, n-1)$ de forma que o caminho forme uma sequência não-crescente de números. Caso não exista um caminho que satisfaça a essas condições, o inteiro -1 deve ser impresso na saída.

Lembre-se de que cada linha da saída, inclusive a última, deve terminar com um "\n".

Exemplos

Entrada	Saída
2 5 4 5 3	1
3 3 2 1 3 1 2 4 2 2	3
4 2 3 2 1 3 2 3 1 1 3 3 2 1 0 0 0	5

Problema F. Robô no Armazém

Entrada de dados:	`entrada padrão`
Saída de dados:	`saída padrão`
Limite de tempo:	3 segundos
Limite de memória:	256 megabytes

Um robô foi programado para organizar as mercadorias de um armazém automatizado. O armazém é dividido em setores dispostos em uma grade de dimensões $n \times m$, e uma das mercadorias precisa ser transportada até uma área específica para ser despachada.

O armazém é representado por uma matriz A de caracteres com n linhas e m colunas, em que cada posição pode representar:

- '.': representa um setor livre, que o robô e a mercadoria podem ocupar.
- '#': representa um setor interditado, que nem o robô nem a mercadoria podem ocupar.
- 'C': representa um setor livre e a posição inicial do robô.
- 'M': representa um setor livre e a posição inicial da mercadoria.
- 'D': representa um setor livre e a área de despacho, onde a mercadoria deve ser entregue.

O robô pode se deslocar para setores vizinhos na horizontal ou na vertical apenas, mas não pode atravessar setores interditados, nem passar por um setor ocupado pela mercadoria, nem sair dos limites do armazém. Para transportar a mercadoria, ele precisa se posicionar ao lado dela e empurrá-la para o setor seguinte. A mercadoria só pode ser empurrada para um setor livre, e o robô não consegue puxá-la, apenas empurrá-la.

Sua tarefa é encontrar o menor número possível **de empurrões** para que a mercadoria chegue ao destino. Os deslocamentos do robô que não movimentam a mercadoria não são contabilizados. Abaixo está um exemplo em que o robô dá 3 empurrões para que a mercadoria chegue à seção de despacho.



Entrada

A primeira linha do caso de teste é composta pelos inteiros n e m, indicando as dimensões da matriz A. Depois, seguem-se n linhas, correspondendo à matriz A que representa o armazém.

Cada linha $0 \leq i \leq n-1$ é composta por uma string com m caracteres

$$\texttt{A}[i][0]\ \texttt{A}[i][1]\ \ldots\ \texttt{A}[i][m-1]$$

representando cada linha de A.

É garantido que

- $2 \leq n, m \leq 200$;
- para cada $0 \leq i \leq n-1$ e $0 \leq j \leq m-1$, $\texttt{A}[i][j]$ é um dos caracteres no conjunto {'C', 'M', 'D', '.', '#'};
- existe exatamente um par $0 \leq i \leq n-1$ e $0 \leq j \leq m-1$ tal que $A[i][j] =$ 'C';
- existe exatamente um par $0 \leq i \leq n-1$ e $0 \leq j \leq m-1$ tal que $A[i][j] =$ 'M'; e
- existe exatamente um par $0 \leq i \leq n-1$ e $0 \leq j \leq m-1$ tal que $A[i][j] =$ 'D'.

Saída

A saída deve conter apenas uma linha contendo a quantidade mínima de empurrões na caixa necessários para levá-la à área de despacho. Caso não exista uma forma de fazer isso, o inteiro -1 deve ser impresso na saída.

Lembre-se de que cada linha da saída, inclusive a última, deve terminar com um "\n".

Exemplos

Entrada	Saída
4 4 D### ..M. .##. ...C	3
4 3 D## .C. ##. ..M	-1
4 4 D..# .#M. ...C	5